CS2 Live Telemetry Engine Setup

https://gemini.google.com/app/db8a2d83269e6179

User prompt: so its not possible to get more informed feed such as coordinate nade coordinates etc?

Response: **From the public HLTV Scorebot socket, no: you cannot get continuous (X, Y, Z) player vectors, crosshair angles, or grenade flight trajectories.**

HLTV's public socket is strictly an **event-and-scoreboard stream**, not a full spatial telemetry feed.

1. What HLTV Actually Knows vs. What It Broadcasts

HLTV receives raw game server logs and GSI streams internally from tournament organizers. However, their public distribution layer deliberately downsamples and filters the stream for two reasons:

- **Bandwidth & Socket Concurrency:** Broadcasting 10 players' 64-tick (X, Y, Z) spatial positions, velocity vectors, and active grenade trajectories to tens of thousands of concurrent web visitors would increase WebSocket egress and infrastructure costs by orders of magnitude.
- **Integrity & Anti-Cheat/Gambling Regulations:** Pushing sub-second player vector telemetry to open web clients with zero broadcast delay creates severe stream-sniping and micro-betting exploit risks.

What You Get vs. What You Do NOT Get from HLTV:

Feature / Metric	Present in HLTV Scorebot?	What it Actually Looks Like
Player (X, Y, Z) Coordinates	❌ **No**	Only team side (CT / TERRORIST) and general map zone if specified in event headers.
Grenade Throw Paths & Detonations	❌ **No**	You only get flash assists (`flashAssist: true`) or HE grenade frags (`weapon: "hegrenade"`). Smoke/molotov trajectories are omitted.
Bomb Location	⚠️ **Site name only**	`"site": "A"` or `"site": "B"`. No exact bomb vector coordinate.
Player Aim / View Vectors	❌ **No**	Pitch, yaw, and crosshair placement are completely absent.
Kill Context	**Partial Flags**	`headShot: true`, `penetrated: true` (wallbang), `throughSmoke: true`.
Player Equipment & Economy	**Full**	Health, armor, helmet, primary weapon, kit, cash, K/D/A.

2. How HLTV's Visual Mini-Map Actually Works

If you watch a match on HLTV's website, you will notice their 2D radar does **not** show real-time smooth player movement like the in-game radar. It only shows static icons pulsing or appearing when an engagement occurs (e.g., an icon flashes when a player dies or plants the bomb at Site A).

HLTV translates events to the web canvas using predefined **callout strings and zones**, not sub-tick coordinate math.

3. How to Get Full Coordinates & Grenade Data

If your project requires vector coordinates (e.g., *"frozen throws a high sky-box smoke from T-Roof landing on Mirage Window, while ZywOo holds an off-angle at Connector"*), you have two alternatives:

Alternative A: The Offline/Replay Path (`demoparser2` on `.dem` files)

Valve's binary `.dem` files contain the full game simulation record. Using the Rust-backed `demoparser2` library, you can extract every sub-tick vector:

```
from demoparser2 import DemoParser

parser = DemoParser("faze_vs_vitality.dem")

# 1. Extract exact player coordinates at every tick or death
deaths_df = parser.parse_event(
    "player_death",
    player_extra=["X", "Y", "Z", "pitch", "yaw", "last_place_name"]
)

# 2. Extract every single grenade trajectory and detonation
grenades_df = parser.parse_event(
    "smokegrenade_detonate",
    player_extra=["X", "Y", "Z"]
)
```

With `.dem` files, you get:

- Exact origin vectors $(X_{\text{origin}}, Y_{\text{origin}}, Z_{\text{origin}})$ and target landing points $(X_{\text{detonate}}, Y_{\text{detonate}}, Z_{\text{detonate}})$.
- In-game nav mesh callout strings (`last_place_name`, e.g., `"Palace"`, `"Apartments"`, `"Connector"`).

Alternative B: Native CS2 Game State Integration (GSI)

When running your own client or observing a match through CSTV, setting `"allplayers_position": "1"` in your `gamestate_integration_*.cfg` forces the CS2 engine to send full vector coordinates via HTTP POST:

```
{
  "allplayers": {
    "player_slot_1": {
      "name": "mONESY",
      "position": "-1420.54, -250.12, 128.00",
      "forward": "0.85, -0.52, 0.05"
    }
  }
}
```

4. What This Means for Your AI Commentator

You do not need continuous coordinate streams to generate compelling live play-by-play casting.

1. **Human Casters Do Not Read Coordinates:** Real esports commentators (like Machine, SPUNJ, or Scrawny) do not say *"ZywOo at coordinate X=-1420, Y=-250 killed frozen"*. They react to:
    - **Who killed whom and with what** (ZywOo `[AK-47] -> frozen`)
    - **Special conditions** (`headshot`, `throughSmoke`, `wallbang`)
    - **Man-advantage swings** (5v4 → 3v2 clutch)
    - **Objective triggers** (Bomb plant on Site A, 1v1 defuse attempt)
    - **Economy context** (Force buy with Galils vs. Full CT AWP setup)
2. **The Practical Prompt Architecture:** The HLTV Scorebot feed provides all of these narrative anchors. You pass the structured state into the LLM:
    ```
    ROUND: 11 | SCORE: 5-5 | FAZE (CT: Full Buy) vs VITALITY (T: Force Buy)
    EVENT (01:14): ZywOo eliminated frozen with AK-47 [HEADSHOT, THROUGH SMOKE]
    EVENT (01:11): apEX eliminated ropz with Galil AR
    OBJECTIVE: Bomb planted on Site A by flamez
    MAN COUNT: Vitality 5v3 FaZe
    ```
    A fine-tuned Llama-3 model turns those discrete flags into high-energy, contextually accurate play-by-play without needing raw Euclidean vectors.

User prompt: the alternatives that you gave, they won't be possible for live matches right?

Response: **For tier-1 and tier-2 professional tournament matches (like IEM, BLAST, or ESL), neither `.dem` parsing nor local GSI is accessible for public live games.**

Understanding the exact technical limitations of each approach clarifies why:

1. Why `.dem` Replay Parsing Cannot Work for Live Games

- **File Finalization:** A `.dem` file is a complete binary recording generated on the tournament server. The game engine writes to this file continuously until the match or map concludes.
- **Release Timing:** Tournament organizers only compress and publish `.dem` files **after the series has completely finished** (usually uploaded to HLTV's database 15–60 minutes after the post-match handshake).
- **Conclusion:** `demoparser2` is strictly an **offline/post-match** tool. It is ideal for mining training data, but it cannot run on a game that is still in progress.

2. Why GSI Cannot Work for Public Pro Matches

- **Host-Only Broadcast:** CS2 Game State Integration only sends data if the machine running the game is configured to push HTTP POST requests to your specific server endpoint.
- **No Public CSTV/GOTV IP Addresses:** In older eras of CS:GO, Valve Majors occasionally published open GOTV relay IPs (`connect <ip>:<port>`). However, in modern professional CS2, tournament organizers (ESL, BLAST, PGL) **strictly lock down server access**:
    - CSTV relays are hosted on private subnets reserved for official broadcast observers, production desks, and authorized partner feeds.
    - Public access is blocked to prevent betting manipulation, server DDoS attacks, and stream-sniping.
- **The Only Way GSI Works Live:** If you are playing in the match, if you run a private community server, or if a tournament platform (like Gamers Club or FACEIT) explicitly grants your account direct CSTV access.

The Reality of Esports Data Pipelines

To build a **live** commentary application without commercial data contracts (which cost thousands of dollars per month), your viable data paths are:

```
                    LIVE PRO CS2 DATA
                           |
       ┌───────────────────┴────────────────────┐
       ▼                                        ▼
[ Commercial APIs ]                    [ HLTV Scorebot ]
• PandaScore / Abios / GRID            • scorebot2.hltv.org
• High cost / restrictive free tiers   • 100% Free
• Mostly high-level kill/score feeds   • Provides Kills, Weapons, Headshots,
                                         Plants, Defuses, & Economy
```

HLTV's live Scorebot socket is the only open, zero-cost data pipe available for active pro tournament matches.

While it omits spatial (X, Y, Z) player coordinates and utility flight paths, it provides every core metric a play-by-play caster needs: who made the frag, which weapon was used, headshot/wallbang/smoke status, bomb site objectives, player health, and full team economy.

Sources

1. FAQ - CSTV - Help Center - Gamers Club

User prompt: so from hltv, we dont have emotional or importance of the kill that commentry would have, such as an important kill, unlikely kill , which generates different emotions from commentator usually, how will we able to do that with hltv data

Response: Professional esports commentators do not generate emotional intensity from raw sound waves or continuous (X, Y, Z) telemetry; they generate it from **contextual drama**.

A frag in Counter-Strike is only hype if the underlying game state makes it so. A 5v1 anti-eco exit kill is boring; a Deagle headshot in a 1v3 with 4 seconds left on the bomb clock generates shouting.

Every single variable needed to compute that drama mathematically is already present in HLTV's raw payloads:

- `Kill`: killer weapon, victim weapon, headshot flag, wallbang flag, smoke flag.
- `scoreboard`: player health, loadout values, player alive counts (CT vs TERRORIST), round score, and team cash.
- `BombPlanted` / `BombDefused`: objective states and ticking timers.

To inject genuine caster hype into your LLM, you build a **Deterministic Heuristic Enrichment Layer (The "Hype Engine")** that sits between HLTV's raw socket and the LLM prompt.

1. Computing The Narrative Context from HLTV Telemetry

Before pushing an event to the LLM, your Python pipeline derives the following tactical dimensions:

Narrative Dimension	How to Derive It from HLTV Data	Caster Emotional Reaction
Weapon Disparity (The "Hero" Factor)	Killer had `deagle` / `cz75a` / `usp_silencer`; Victim had `awp` / `ak47`.	*"HE WINS IT WITH JUST A PISTOL! WHAT AN UPSET!"*
Mechanical Difficulty	`headShot: true` + `throughSmoke: true` or `penetrated: true` (wallbang).	*"BLIND THROUGH THE SMOKE! HOW DOES HE FIND THAT?!"*
Man-Advantage Delta (The Clutch State)	Count alive players in `scoreboard`: `1v3`, `1v2`, `2v5`.	*"IT'S DOWN TO 5'SMPLE. CAN HE PULL OFF THE IMPOSSIBLE?"*
Time & Objective Urgency	`bombPlanted: true` + defuse kit status (`hasDefusekit: false` on CTs).	*"NO KIT! HE HAS TO STICK IT! EVERY MILLISECOND COUNTS!"*
Economy Stakes	Team bank $< \$2000$ (Loss bonus break / Reset round).	*"THIS IS A TOURNAMENT-DECIDING ECO ROUND!"*
Tournament Leverage	Match score is `12-11` or `12-12` (Overtime / Match Point).	*"MATCH POINT PRESSURE! THERE IS ZERO MARGIN FOR ERROR!"*

2. The Heuristic Enrichment Pipeline

Here is an architectural Python engine that ingests the raw HLTV socket frames, calculates an **Importance Score** ($0.0 \rightarrow 10.0$), assigns an **Emotional Tone**, and tags the kill with high-level tactical tags:

```
from dataclasses import dataclass
from typing import Dict, Any, List

WEAPON_TIERS = {
    "glock": 1, "usp_silencer": 1, "p250": 1, "deagle": 2, "cz75a": 2,
    "mp9": 2, "mac10": 2, "galilar": 3, "famas": 3,
    "ak47": 4, "m4a1_silencer": 4, "m4a4": 4, "awp": 5
}

@dataclass
class EnrichedCombatEvent:
    killer: str
    victim: str
    weapon: str
    importance_score: float  # 1.0 to 10.0
    emotional_tone: str      # DISBELIEF, HIGH_ENERGY_HYPE, TACTICAL_TENSION, ROUTINE
    narrative_tags: List[str]
    man_advantage: str

class NarrativeEngine:
    def __init__(self):
        pass

    def evaluate_kill(self, kill_event: Dict[str, Any], scoreboard: Dict[str, Any]) -> EnrichedCombatEvent:
        killer = kill_event.get("killerNick", "Unknown")
        victim = kill_event.get("victimNick", "Unknown")
        weapon = kill_event.get("weapon", "ak47").lower()
        headshot = kill_event.get("headShot", False)
        smoke = kill_event.get("throughSmoke", False)
        wallbang = kill_event.get("penetrated", False)

        # 1. Calculate Man Count (e.g., 5v4, 1v3)
        ct_alive = sum(1 for p in scoreboard.get("CT", []) if p.get("alive"))
        t_alive = sum(1 for p in scoreboard.get("TERRORIST", []) if p.get("alive"))
        killer_side = kill_event.get("killerSide", "CT")

        killer_team_count = ct_alive if killer_side == "CT" else t_alive
        victim_team_count = t_alive if killer_side == "CT" else ct_alive
        man_adv_str = f"{killer_team_count}v{victim_team_count}"

        # 2. Base Importance Calculation
        score = 3.0
        tags = []

        # First Blood (Entry Kill)
        if (ct_alive + t_alive) == 9:
            score += 2.0
            tags.append("FIRST_BLOOD")

        # Clutch Situation (1vX)
        if killer_team_count == 1 and victim_team_count >= 1:
            score += 3.5
            tags.append(f"1v{victim_team_count + 1}_CLUTCH_ATTEMPT")
```

```
        # Difficulty & Flukes
        if smoke:
            score += 2.0
            tags.append("BLIND_SMOKE_KILL")
        if wallbang:
            score += 1.5
            tags.append("WALLBANG")
        if weapon == "deagle" and headshot:
            score += 2.0
            tags.append("JUAN_DEAG")

        # Weapon Disparity Check (Eco weapon killing an AWP/Rifle)
        victim_weapon = "ak47"
        for team in ("CT", "TERRORIST"):
            for p in scoreboard.get(team, []):
                if p.get("name") == victim:
                    victim_weapon = p.get("primaryWeapon", "ak47")

        killer_tier = WEAPON_TIERS.get(weapon, 3)
        victim_tier = WEAPON_TIERS.get(victim_weapon, 3)
        if victim_tier - killer_tier >= 2:
            score += 2.5
            tags.append("UPSET_ECO_PUNISH")

        # Match Leverage (Match Point / Overtime)
        round_num = scoreboard.get("currentRound", 1)
        ct_score = scoreboard.get("ctScore", 0)
        t_score = scoreboard.get("tScore", 0)
        if ct_score >= 12 or t_score >= 12:
            score += 1.5
            tags.append("HIGH_LEVERAGE_ROUND")

        # 3. Classify Emotional Tone
        score = min(10.0, score)
        if score >= 8.5:
            tone = "DISBELIEF_AND_ROARING_HYPE"
        elif score >= 6.5:
            tone = "HIGH_ENERGY_CAST"
        elif score >= 4.5:
            tone = "TACTICAL_INTENSITY"
        else:
            tone = "ROUTINE_PLAY_BY_PLAY"

        return EnrichedCombatEvent(
            killer=killer,
            victim=victim,
            weapon=weapon,
            importance_score=score,
            emotional_tone=tone,
            narrative_tags=tags,
            man_advantage=man_adv_str
        )
```

3. How the LLM Interprets It

Instead of passing bare logs like `ZywOo killed frozen with deagle`, you feed the LLM the enriched structural context:

```
[SYSTEM]: You are a tier-1 esports caster (like Scrawny or Machine). Match the exact emotional intensity specified in the tags.

[CONTEXT]:
- Match State: Round 24 (Score: 12-11, MATCH POINT)
- Situation: 1v2 Clutch Attempt by ZywOo (Low HP: 14)
- Event: ZywOo eliminated frozen with deagle [HEADSHOT, BLIND_SMOKE_KILL]
- Hype Score: 9.5 / 10.0
- Tone Directive: DISBELIEF_AND_ROARING_HYPE

[COMMENTARY TARGET]:
```

What the LLM Outputs:

> *"ARE YOU KIDDING ME?! ZywOo with a Desert Eagle, on fourteen HP, blind through the smoke on match point! Frozen is deleted! It's down to a 1v1—the Major is hanging by a thread!"*

By turning raw scoreboard counters and weapon names into explicit drama and leverage metrics, your fine-tuned model knows when to remain measured and tactical, and when to deliver high-energy casting.

User prompt: will we have hp like that? like from hltv , whenever it updates on hltv

Response: Yes, player HP is explicitly present in the data, but understanding **how and when** HLTV transmits it is critical for calculating drama and clutch states.

How HLTV Transmits Player HP

HLTV sends health data through the periodic **`scoreboard`** snapshot.

Every player entry inside the CT and TERRORIST arrays has an exact integer field for hp:

```
{
  "name": "ZywOo",
  "alive": true,
  "hp": 14,
  "armor": 100,
  "helmet": true,
  "money": 4600,
  "primaryWeapon": "deagle"
}
```

Update Cadence: How Often Does It Refresh?

HLTV does **not** emit a micro-packet for every single bullet that connects (there is no standalone `bulletHit` event on the public socket). Instead, HP updates through two mechanisms:

1. **The `scoreboard` Refresh Cycle:** HLTV broadcasts the full scoreboard state every few seconds (and whenever significant state shifts occur), updating the live hp value for all 10 players.
2. **The `Kill` Event:** When an elimination occurs, the victim's HP drops to `0` and `alive` flips to `false`.

How to Maintain Real-Time HP State in Python

Because the combat log (`Kill`, `Assist`) and the game state (`scoreboard`) arrive on separate socket channels, keep an in-memory dictionary tracking player health across the match:

```
class MatchStateManager:
    def __init__(self):
        # Maps player_nickname -> current_HP
        self.player_hp = {}
        self.player_alive = {}

    def update_from_scoreboard(self, scoreboard_payload: dict):
        """Refreshes exact HP values whenever HLTV broadcasts a scoreboard."""
        for side in ("CT", "TERRORIST"):
            for player in scoreboard_payload.get(side, []):
                nick = player.get("name")
                self.player_hp[nick] = player.get("hp", 100)
                self.player_alive[nick] = player.get("alive", True)

    def handle_kill(self, kill_payload: dict):
        """Instantly zeroes the victim's HP upon a kill event."""
        victim = kill_payload.get("victimNick")
        killer = kill_payload.get("killerNick")

        self.player_hp[victim] = 0
        self.player_alive[victim] = False

        # Access the killer's surviving HP for clutch evaluation
        killer_hp = self.player_hp.get(killer, 100)
        return killer_hp
```

Whenever an elimination or clutch occurs, this in-memory lookup provides the player's exact health (e.g., `killer_hp == 14`), allowing your narrative engine to trigger low-health hype directives for the commentary prompt.

Sources

1. HLTV has changed its websocket endpoint · Issue #42 · GitHub

---

User prompt: I would not want to do. manual if else like you gave, I would want model fine tuned to infer important informations and generate the text accordingly based on information we provide,



Relying entirely on end-to-end representation learning rather than hard-coded heuristics is the correct machine learning approach. A language model with multi-head causal attention is inherently capable of computing relational dependencies—such as correlating `weapon: deagle`, `headShot: true`, and an asymmetrical alive-player count—directly through its learned weight projections.

When you fine-tune an autoregressive decoder model (like Llama-3 or Mistral) on paired match telemetry and authentic human caster transcripts, the self-attention heads learn the underlying distributions of:

1. **Implicit Stakes Representation:** Attending to player counts, economy deficits, and round counts simultaneously to modulate vocabulary intensity and exclamation density.
2. **Dynamic Vocabulary Selection:** Shifting from steady, descriptive prose during 5v5 setups to high-cadence, punchy verbs during site executes or 1vX situations.
3. **Implicit Weapon Valuations:** Learning that an unarmored player eliminating an AWP with a starter pistol represents a statistical anomaly that demands heightened emphasis.

1. The Raw-to-Token Representation Schema

To allow the neural network to infer context without intermediate heuristic code, feed it structured, low-level telemetry serializations (JSON or tabular key-value blocks) as the user turn, mapping directly to the caster's raw transcribed words as the assistant target.

Training Pair Schema (ChatML / JSONL)

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are a professional Counter-Strike 2 play-by-play caster. Deliver high-energy, context-aware commentary matching the tactical stakes and action in the provided game telemetry."
    },
    {
      "role": "user",
      "content": "MAP: de_mirage | ROUND: 24 | SCORE: CT 12 - 11 T\nCT_ALIVE: [{\"name\": \"frozen\", \"hp\": 100, \"weapon\": \"m4a1_silencer\", \"has_kit\": true}]\nT_ALIVE: [{\"name\": \"ZywOo\", \"hp\": 14, \"weapon\": \"deagle\", \"armor\": 0}]\nOBJECTIVE: C4 planted at A (32s remaining)\nRECENT_EVENTS:\n- 00:03.2 ago: ZywOo eliminated broky (m4a1_silencer
    },
    {
      "role": "assistant",
      "content": "ZYWOO HAS DONE THE IMPOSSIBLE! ON FOURTEEN HP, WITH JUST A DESERT EAGLE, HE CRACKS IT WIDE OPEN THROUGH THE SMOKE AND WINS THE CLUTCH FOR VITALITY!"
    }
  ]
}
```

By providing only the raw state fields (`hp`, `score`, `weapon`, `smoke`, `headshot`, and the event delta), the model learns:

- $Score = (12, 11) \implies$ Match point context (high leverage).
- $14\ \text{HP}$ vs $100\ \text{HP} \implies$ Fragile survival under pressure.
- `deagle` against two `m4a1_silencer` rifles $\implies$ Severe equipment disparity.

2. How the Model Learns Tone Without Handcrafted Rules

In causal language modeling, the cross-entropy loss function:

$$\mathcal{L} = -\sum_{t=1}^{T} \log P_\theta(y_t \mid y_{<t}, X_{\text{telemetry}})$$

forces the parameters θ to minimize divergence between predicted tokens and actual caster reactions.

During backpropagation:

- When $X_{\text{telemetry}}$ contains normal trades in an anti-eco round, the target text in the training data consists of routine descriptive commentary. Gradients adjust the weights toward controlled sentence structures and lower-entropy token choices.
- When $X_{\text{telemetry}}$ contains 1vX situations, low HP, or high-tier weapon upsets, the target commentary contains short exclamations, proper names emphasized with dramatic verbs (*"annihilates"*, *"snatches"*, *"demolishes"*), and punctuation flags like exclamation marks.
- The attention layers automatically compute the interaction terms between disparate features (e.g., $W_Q \cdot$ `round_state` interacting with $W_K \cdot$ `hp_remaining`) without requiring hand-coded rule trees or static mathematical thresholds.

3. The End-to-End Pipeline

```
[ HLTV Raw Socket Frame ]
          | (Pure JSON payload: Kills, scoreboard, objective state)
          ▼
[ Ingestion Serializer ]
          | (Deterministic string formatting: key-value/markdown dump)
          ▼
[ Fine-Tuned LLM (Llama-3-8B / QLoRA) ]
          | (Implicitly computes leverage, rarity, and emotion via self-attention)
          ▼
[ Live Streamed Commentary Output ]
```

This design keeps your backend lean:

1. No manual rule tuning when the weapon meta shifts.
2. Zero custom heuristic scoring thresholds that risk misinterpreting unique plays.
3. The model handles all narrative pacing, tactical inference, and emotional inflection directly from the data distribution.

User prompt: so we will do 2 fine tunings right? first based on the data that we get from hltv, raw information and how to infer important info from there., and secondly for commentry generation which we do from data of YouTube captions or something

Response: **No, you do not need two separate fine-tunings, nor should you run two.**

Doing this in two separate fine-tuning stages introduces significant engineering drawbacks:

1. **Compounding Latency:** Running two sequential LLM inferences for every game event doubles token generation time ($t_{\text{total}} = t_{\text{stage1}} + t_{\text{stage2}}$), causing live commentary to lag behind the broadcast.
2. **Error Cascades:** If the first model misinterprets an economy state or misses an important frag, the second model receives bad inputs and generates flawed commentary.
3. **Redundant Training:** Modern instruction-tuned models (like Llama-3-8B or Mistral-7B) already possess reasoning and natural language generation capabilities. They do not need a dedicated model just to extract or reformat key-value pairs.

Instead, the standard architecture is an **End-to-End Single-Stage Fine-Tuning** pipeline, with a clean structural separation during training.

The Architecture: One Model, End-to-End Mapping

Rather than training two models, you train **one model** on a unified dataset where:

- **The Input (X):** The raw, structured match telemetry (player health, active weapons, round scores, recent frags).

- **The Target (`Y`):** The authentic caster transcript extracted and aligned from YouTube broadcast VODs.

```
┌──────────────────────────────┐
│    Raw Telemetry Block (X)   │
│ Score: 12-11 | Round 24      │
│ ZywOo: 14 HP, Deagle         │
│ frozen: 100 HP, M4A1-S       │
│ Event: ZywOo eliminated frozen │
│        [Headshot, Smoke]     │
└──────────────┬───────────────┘
               │
               ▼
     [ Single Fine-Tuned LLM ]  <--- (Learns BOTH importance reasoning
               │                      AND commentary style simultaneously)
               ▼
┌──────────────────────────────┐
│    Caster Commentary (Y)     │
│ "ZYWOO WITH THE DEAGLE       │
│  THROUGH THE SMOKE! ON 14 HP! │
│  VITALITY STEAL MATCH POINT!" │
└──────────────────────────────┘
```

During gradient descent, the causal attention heads learn to connect the two concepts directly:

- When the input contains high-leverage data points (`hp: 14`, `weapon: deagle` vs rifle, `score: 12-11`), the loss function trains the model to emit urgent verbs, exclamations, and high-intensity phrasing.
- When the input contains low-stakes data points (a routine 5v1 anti-eco exit kill), the model generates calm, analytical sentences.

If You Want Explicit Analytical Reasoning: Chain-of-Thought (CoT)

If you want the model to explicitly reason about the importance of a play before generating the casting line, you still do not need two models. You use **Chain-of-Thought (CoT) output structuring within a single model**.

In this design, the model generates an internal `<analysis>` tag before outputting the public `<commentary>`:

```
{
  "messages": [
    {
      "role": "user",
      "content": "MAP: Mirage | ROUND: 24 | SCORE: CT 12 - 11 T\nCT_STATE: [{\"name\": \"frozen\", \"hp\": 100, \"weapon\": \"m4a1_silencer\"}]\nT_STATE: [{\"name\": \"ZywOo\", \"hp\": 14, \"weapon\": \"deagle\"}]\nRECENT_KILL: ZywOo eliminated frozen with deagle [headshot: true, smoke: true]"
    },
    {
      "role": "assistant",
      "content": "<analysis>Severe weapon mismatch: Deagle vs M4A1-S. ZywOo surviving on critical 14 HP in a 1v1 clutch. Round 24 represents match point leverage. Elimination through smoke implies high mechanical rarity. Tone must be peak disbelief and hype.</analysis>\n<commentary>ZYWOO HAS DONE IT AGAIN! ON FOURTEEN HP WITH A DESERT EAGLE, BLIND THROUGH THE S
    }
  ]
}
```

Why This Is Superior to Two Separate Models

- **Single Deployment Footprint:** You maintain, quantize, and host only one set of LoRA adapter weights (fitting within free T4 GPU / Ollama memory limits).
- **Zero Inter-Service Overhead:** You eliminate an entire HTTP/IPC hops between multiple inference microservices.
- **Streamlit UI Integration:** Your web app can parse the output tokens on the fly: display `<commentary>` in the main live broadcast box, and route `<analysis>` into a side tactical drawer for viewers who want tactical breakdowns.

What the Two Phases Actually Refer To

You likely thought of "two stages" because data preparation involves two distinct extraction steps:

1. **Step 1 (Data Alignment / Offline ETL):**
   - Collect raw telemetry from `.dem` replay files (or live HLTV event logs).
   - Collect caster speech transcripts from YouTube match VODs using Whisper.
   - Merge them by timestamp into pairs: `(Telemetry_Window, Caster_Transcript)`.
2. **Step 2 (The Fine-Tuning Run):**
   - Take that merged dataset and run **a single QLoRA fine-tuning session** on Llama-3-8B or Mistral-7B.

This produces a single, low-latency, free-to-run model that consumes raw HLTV feeds and directly outputs live commentary.

Sources

1. Finetuning Fun. How LLMs Are Built: Two Main Stages | by Yin

---

User prompt: then I will have to get YouTube captions which one to match with feed from hltv match data, that too for multiple matches (more the marrier) ?

Response: Yes, you need data from multiple matches to train the model properly. However, **manually downloading and aligning dozens of full YouTube VODs with live HLTV match data by hand is practically impossible and inefficient.**

The primary reason: **HLTV does not store historical real-time socket events forever**. HLTV's public Scorebot is live-only; once a match finishes, the microsecond WebSocket stream disappears, leaving only high-level post-match box scores.

To build a high-quality dataset of hundreds or thousands of commentary-telemetry pairs, you must use an **automated offline extraction pipeline**.

1. Where Does the Game Data Actually Come From?

Instead of trying to record HLTV's live feed across weeks, you use **Valve's `.dem` replay files**:

- Every single tier-1 tournament (BLAST, ESL, IEM, Majors) has its complete pro `.dem` replays archived publicly on HLTV for free download.
- A single `.dem` file contains every single tick of the game: exact health changes, weapons, player positions, headshots, and wallbangs.
- Parsing one `.dem` file with `demoparser2` takes roughly **3 to 5 seconds** in Python and produces the exact same event schemas (kills, plants, scoreboard) that HLTV pushes over the socket.

2. How Many Matches Do You Actually Need?

For QLoRA fine-tuning an already capable language model (like `Llama-3.2-3B-Instruct` or `Llama-3-8B-Instruct`):

Matches Processed	Clash/Event Spans Extracted	Quality & Capability
1 – 2 Matches (e.g., 1 Bo3 series)	~150 – 300 samples	Proof of concept. Learns formatting, but hallucinates non-standard situations.
5 – 10 Matches	**1,500 – 3,000 samples**	**Sweet spot.** Captures authentic caster vocabulary, clutches, eco rounds, and high-tension tone shifts without overfitting.
50+ Matches	15,000+ samples	Diminishing returns. Slower training, risk of overfitting to specific casters' idiosyncratic verbal tics.

5 to 10 full matches (or roughly two tournament playoff weekends) are more than enough to yield thousands of diverse combat interactions.

3. The 3-Step Automated Alignment Pipeline

Instead of manually scrubbing video timelines, the pipeline automates extraction, transcription, and alignment:

```
[ Pro Tournament VOD (YouTube) ]              [ Match Demo (.dem) ]
               │                                       │
       (yt-dlp: mono wav)                      (demoparser2: Rust)
               ▼                                       ▼
 [ Whisper with Word Timestamps ]            [ Structured Event Ticks ]
 (e.g., "01:24.5: ZywOo finds frozen...")    (e.g., Tick 12840: Kill ZywOo -> frozen)
               └──────────────────┬──────────────────┘
                                  ▼
                    [ Temporal Anchor Calibration ]
                      (Find At using Round 1 start)
                                  ▼
                    [ Training Samples (JSONL) ]
```

Step A: Audio & Word-Level Transcription

You do not use default YouTube auto-captions (which omit punctuation, censor profanity, and mangle player handles like *blameF*, *ropz*, or *m0NESY*).

Download the audio track via `yt-dlp` and pass it to **Whisper** (or `whisperX`) with `word_timestamps=True`:

```
{
  "start": 84.2,
  "end": 88.6,
  "text": "ZYWOO CRACKS IT OPEN THROUGH THE SMOKE! What a shot!"
}
```

Step B: Anchor Alignment (Calculating Δt)

Broadcast streams always have an arbitrary delay (countdown screens, sponsor segments). To find the offset Δt:

1. Locate the exact game tick when **Round 1 begins** in the `.dem` file ($T_{\text{demo_R1}}$).
2. Scan the first 10 minutes of the Whisper transcript for the phrase *"Pistol round"* or *"Round 1"* ($T_{\text{whisper_R1}}$).
3. The global time offset is:

$$\Delta t = T_{\text{whisper_R1}} - T_{\text{demo_R1}}$$

Step C: Window Slicing into Training Pairs

Once synchronized, loop through the demo events:

- When a clash occurs at t_{demo}, slice the game state into your telemetry prompt.
- Extract the caster speech from the Whisper transcript in the window $[t_{\text{demo}} + \Delta t,\ t_{\text{demo}} + \Delta t + 8.0\text{s}]$.
- Write the `(telemetry, transcript)` record to `train.jsonl`.

The Practical Alternative: A High-Quality Synthetic Seed Dataset

If downloading multi-gigabyte VODs and running Whisper audio transcriptions feels like too heavy of an infrastructure lift just to start training, there is a proven industry alternative used in esports ML:

1. **Extract 5,000 real game clashes** from `.dem` files or HLTV match histories.
2. Feed those raw telemetry blocks to a frontier model (like GPT-4o or Claude 3.5 Sonnet) using a strict system prompt loaded with authentic caster transcripts and esports slang dictionaries (*"Cast this raw CS2 play like Anders and Sco*mmy*").
3. This creates a clean, zero-noise, perfectly aligned training dataset in minutes without having to troubleshoot audio desync issues.
4. Fine-tune your local Llama-3 model on this dataset using QLoRA.

Either approach produces the identical result: a compact, fine-tuned model ready to consume your live HLTV socket feed and stream real-time commentary.

Sources

1. MatchTime - Haoning Wu

User prompt: if I use valves dem to train, which will be different and more detailed than hltv's data right? so when we infer live won't there be mismatch

Response: Yes, if you train directly on the full raw telemetry of a `.dem` file, you will create a severe **train-test distribution shift** (data mismatch), causing the model to hallucinate or underperform during live inference.

A `.dem` file contains continuous 64-tick sub-tick vectors, (X, Y, Z) player coordinates, exact pitch/yaw angles, crosshair placement, velocity, and millisecond grenade trajectories. If your model learns to look for spatial tokens like: `"position": [-1420.5, 320.1], "pitch": -12.4, "utility_detonated": "flashbang"` it will expect those features at test time. When your live production inference feeds it HLTV's leaner, event-driven socket stream (which lacks those fields), the model's attention mechanism degrades.

The Standard Fix: Telemetry Masking / Projection Downsampling

In machine learning, when your offline training data is richer than your real-time production features, the standard solution is **Projection Downsampling** (or feature masking).

You do **not** train the model on the full `.dem` stream. Instead, you write an extraction filter in Python that discards the extra data and retains **only the exact subset of features that HLTV transmits live**.

```
[ Valve .dem File (Complete Ground Truth) ]
  ├── (X, Y, Z) Vectors              ──► [ DISCARD / STRIP ]
  ├── Sub-tick Aim Angles            ──► [ DISCARD / STRIP ]
  ├── Continuous Velocity            ──► [ DISCARD / STRIP ]
  │
  ├── Kills & Headshots              ──► [ KEEP ]
  ├── Weapons Used                   ──► [ KEEP ]
  ├── Wallbang & Smoke Flags         ──► [ KEEP ]
  ├── Player HP & Alive Status       ──► [ KEEP ]
  ├── Bomb Plant / Defuse Events     ──► [ KEEP ]
  └── Round Score & Team Economy     ──► [ KEEP ]
                                          │
                                          ▼
                        [ Downsampled Intermediate Schema ]
                                          │
                                          ▼
                          Matches HLTV Live Ingestion 1:1
```

By enforcing a shared intermediate schema, the model never knows whether a training sample came from a 2024 Valve demo replay or a live 2026 HLTV socket stream.

Implementing the Shared Schema

Define a single canonical data structure (for example, using Pydantic or a standard Python dataclass) that acts as the contract for both environments:

```
from dataclasses import dataclass, asdict
from typing import List, Optional

@dataclass
class PlayerState:
    name: str
    hp: int
    alive: bool
    weapon: str
    has_kit: bool = False
    money: int = 0

@dataclass
class KillEvent:
    killer: str
    victim: str
    weapon: str
    headshot: bool
    through_smoke: bool
    penetrated: bool  # wallbang
    assister: Optional[str] = None

@dataclass
class MatchFrame:
    map_name: str
    round_number: int
    ct_score: int
    t_score: int
    ct_players: List[PlayerState]
    t_players: List[PlayerState]
    events: List[KillEvent]
```

1. At Training Time (from `.dem` via `demoparser2`):

Extract the raw tick events and discard the spatial vectors, keeping only the fields defined in `MatchFrame`:

```
# demoparser2 extraction
```

```
def extract_training_frame(demo_parser, tick) -> MatchFrame:
    # 1. Extract death events at this tick
    deaths = demo_parser.parse_event("player_death")
    # 2. Extract player inventories and HP (discarding position X, Y, Z)
    # 3. Construct and serialize strictly to MatchFrame
    ...
```

2. At Production Inference Time (from HLTV Scorebot):

Map the incoming HLTV socket dictionaries into the identical `MatchFrame`:

```
# HLTV socket ingestion
def extract_live_frame(hltv_log, hltv_scoreboard) -> MatchFrame:
    # 1. Parse raw log['Kill']
    # 2. Parse scoreboard['CT'] and scoreboard['TERRORIST']
    # 3. Construct and serialize strictly to MatchFrame
    ...
```

What the Tokenized Prompt Looks Like

Because both inputs pass through the same serializer, the textual representation fed to the LLM during training and live inference is identical:

```
MAP: de_mirage | ROUND: 18 | SCORE: CT 10 - 7 T
CT_ALIVE:
- broky | HP: 100 | WEAPON: awp | KIT: true
- frozen | HP: 64 | WEAPON: m4a1_silencer | KIT: true
T_ALIVE:
- ZywOo | HP: 14 | WEAPON: deagle | KIT: false
RECENT_EVENTS:
- ZywOO eliminated ropz with deagle [headshot: true, smoke: true, wallbang: false]
```

Using this approach:

1. **Zero Domain Shift:** The model trains on the exact level of abstraction it encounters live.
2. **Abundant Training Data:** You can parse thousands of pro `.dem` files archived over years of tournaments to build your training set, without writing ad-hoc scraping scripts.
3. **No Redundant Modeling:** The LLM focuses its weights on casting cadence, player names, weapon dynamics, and round stakes rather than trying to interpret raw 3D geometry coordinates.

User prompt: is it possible to get like stats of each players, that we usually have like k/d/a ,money, damage after each round from hltv ? I would like to like infer and highlight some info in between rounds like some crazy play during the round, or important stat, something.

Response: Yes. The `scoreboard` payload broadcast by HLTV's Scorebot provides player-by-player accounting, enabling inter-round analysis and highlighting crazy plays.

1. What HLTV Gives Directly vs. What You Derive

Metric	Available Directly in Scorebot?	Field Location
Kills, Deaths, Assists (K/D/A)	**Yes** (Cumulative)	`kills`, `deaths`, `assists` per player
Current Cash ($)	**Yes**	`money` per player
Damage Dealt (ADR)	⚠️ **Indirect / Not as a raw stat**	Calculated from HP drops or final box scores
Round Frags (1k, 2k, 3k, Ace)	**Yes** (Derivable)	Difference in `kills` between round starts
Clutch Plays	**Yes** (Derivable)	Last alive player eliminating the remaining enemies

Inside each `scoreboard` event, both the `CT` and `TERRORIST` lists contain a full stats dictionary for every player:

```
{
  "name": "ZywOo",
  "alive": true,
  "hp": 84,
  "money": 4250,
  "kills": 16,
  "deaths": 7,
  "assists": 4,
  "score": 38,
  "primaryWeapon": "ak47"
}
```

2. How to Track Round-Over-Round Stats

To compute round-specific accomplishments—such as **who got a multi-kill**, **who carried the round**, or **economy broken**—snapshot player stats when `roundStart` fires, and compare them when `roundEnd` occurs.

```
class RoundTracker:
    def __init__(self):
        self.start_snapshot = {}
        self.round_events = []

    def on_round_start(self, scoreboard: dict):
        """Snapshots player stats at the beginning of the round."""
        self.start_snapshot.clear()
        self.round_events.clear()

        for side in ("CT", "TERRORIST"):
            for p in scoreboard.get(side, []):
                self.start_snapshot[p["name"]] = {
                    "kills": p.get("kills", 0),
                    "deaths": p.get("deaths", 0),
                    "money": p.get("money", 0),
                    "side": side
                }

    def record_kill(self, killer: str, victim: str, weapon: str, headshot: bool):
        self.round_events.append({
            "killer": killer,
            "victim": victim,
            "weapon": weapon,
            "headshot": headshot
        })

    def generate_round_recap(self, end_scoreboard: dict, winner_side: str) -> dict:
        """Computes who went off during the round."""
        recap = {
            "mvp_candidates": [],
            "multi_kills": {},
            "eco_damage": {}
        }

        # 1. Calculate kills earned specifically in this round
        for side in ("CT", "TERRORIST"):
            for p in end_scoreboard.get(side, []):
                name = p["name"]
                init = self.start_snapshot.get(name, {"kills": 0, "money": 0})
                round_kills = p.get("kills", 0) - init["kills"]

                if round_kills >= 2:
                    recap["multi_kills"][name] = round_kills

                # Check for major bank collapse (force buy gone wrong)
                if init["money"] > 3500 and p.get("money", 0) < 1500 and not p.get("alive"):
                    recap["eco_damage"][name] = "Bank reset / broken"

        return recap
```

3. How to Present This Data to the Model

During the 15-second freeze time between rounds, pass this compiled statistical block to the LLM.

Instead of manual scripts dictating the commentary, present the factual changes to the model and let it handle the analysis:

```
ROUND_RECAP:
- Round 14 Winner: TERRORISTS (Target Bombed)
- Score: FaZe (CT) 7 - 7 Vitality (T)

PER-ROUND PERFORMANCE:
- ZywOo: 3 kills this round (AK-47), survived with 84 HP. Total K/D: 16-7.
- ropz: 0 kills, died early to entry duel. Total K/D: 8-11.
- FaZe Loss Bonus: Maxed out ($3,400 incoming), but bank completely depleted ($1,400 avg).

ROUND EVENTS HIGHLIGHT:
- ZywOo eliminated karrigan (Headshot)
- ZywOo eliminated rain (Trade)
- ZywOo eliminated frozen (Site Anchor)
```

What the Fine-Tuned LLM Outputs

Because the model was trained on pro caster commentary, it recognizes the significance of a 3k round entry and economy reset, generating analysis like:

> *"Vitality lock up the half at seven apiece, and make no mistake—that round belonged entirely to Mathieu 'ZywOo' Herbaut. A devastating triple-kill onto the A-site defense leaves FaZe completely destitute heading into the switch. With their bank balance in shambles, FaZe will have to make a miraculous pistol round happen on the attack."*

User prompt: like I want to say is, will it be able to spot something different, either from the round that happened just now or from stats and highlight it during the buy of the two rounds

Response: Yes. This is where an autoregressive language model genuinely shines over static templates.

During the standard 15-to-20-second **freeze time (buy phase)** between rounds, professional casters do not talk about bullet sprays. They shift into analytical desk mode: they point out anomalous stats, a single player going nuclear, economy blunders, or strange buy behaviors.

Because the model processes the match history through attention, it spots statistical and contextual anomalies automatically—provided the prompt surfaces both the **just-concluded round's events** and the **upcoming buy phase state**.

Anomalies the Model Can Spot from HLTV Data

The HLTV Scorebot exposes cumulative K/D/A, round money, loss bonus counters, loadouts, and the exact sequence of frags in that round. By feeding the delta between rounds, the model naturally identifies:

1. **The Outlier Multi-Kill / Carry Round**
   - *Telemetry Delta:* Player has 3 or 4 kills in the round just finished while the other 4 teammates had 0.
   - *Model Analysis: "That round was pure heroics from ZywOo—a solo quad-kill onto A-site to drag Vitality kicking and screaming to round six."*
2. **The "Stat Leader Falling Silent" / Underperformance**
   - *Telemetry Delta:* Star rifler sits at `0 kills, 6 deaths` over the last 6 rounds, or the team's primary AWPer died first three rounds in a row with zero impact frags.
   - *Model Analysis: "Keep your eyes on frozen here during the freeze time. He's posted a zero-kill board across the last four rounds, and FaZe desperately need their anchor to wake up."*
3. **Economy Mistakes / Risky Buys ("Hero Buy" vs. "Full Eco")**
   - *Telemetry Delta:* 4 players on a team have $1,400 and no armor, but 1 player bought an AWP or Deagle + Armor and emptied their bank to $0.
   - *Model Analysis: "A bizarre decision in the buy phase from FaZe! Four players are running glocks on an eco, but broky forces up a lone AWP with no armor. If he drops early, their bank is completely broken for the next two rounds."*
4. **The Loss-Bonus Break (The "Reset" Round)**
   - *Telemetry Delta:* Team was on a 3-round win streak, loses one round, and their bank drops below $1,800 across all five players.
   - *Model Analysis: "And that loss reset is catastrophic! Vitality's money gets wiped clean. They cannot even afford a full utility set heading into round 11."*
5. **Pistol Upgrade Upsets**
   - *Telemetry Delta:* Round won with `cz75a` and `deagle` kills against full rifle setups.
   - *Model Analysis: "How did they let that slip? G2 drop a full-buy round to five upgraded pistols. The tactical discipline completely collapsed."*

Structuring the Prompt for the Buy Phase

During active combat, your prompt is short and focused on kills. At the `roundEnd` signal, you invoke a dedicated **`INTER_ROUND_ANALYSIS`** prompt formatted like this:

```
SYSTEM: You are an analytical esports color commentator. Highlight surprising statistical trends, economy anomalies, or standout individual performances from the round that just concluded, and set the stakes for the upcoming round.

MATCH: Vitality (T) vs FaZe (CT) | MAP: Mirage
SCORE: 6 - 5 | ROUND COMPLETED: 11 | CURRENT PHASE: FREEZE TIME (BUY PHASE)

JUST COMPLETED ROUND SUMMARY:
- Winner: Vitality (Elimination)
- Round Frags:
  * ZywOo: 3 kills (AK-47)
  * Spinx: 2 kills (Galil AR)
  * FaZe: 0 kills (Flawless round for Vitality)

SCOREBOARD & ECONOMY TRANSITION:
- Vitality Bank (Incoming): [$5,200, $4,800, $6,100, $3,900, $4,400] -> Healthy Full Buy
- FaZe Bank (Incoming): [$1,500, $1,400, $1,900, $1,400, $1,600] -> Broken Bank
- Cumulative Standouts:
  * ZywOo: 18-4 K/D (142 ADR pacing, +14 diff)
  * ropz: 2-8 K/D (0 kills in last 4 rounds)

NEW ROUND BUY OBSERVATION:
- FaZe Loadouts: 4x USP-S (No Armor), but broky purchased Scout + Light Armor ($100 remaining).
```

What the Fine-Tuned Model Generates

> *"Vitality make it look effortless with a clean, flawless round—and that is largely thanks to ZywOo, who is simply operating on another planet right now, sitting at eighteen and four. But look across at the FaZe side of the desk during this freeze time: their economy is utterly shattered. Ropz has gone completely cold over the last four rounds, and broky is gambling his last remaining dollars on a lone scout. If FaZe can't find an opening pick here, this half is going to slip completely out of reach."*

By giving the model the **delta** between the old round and the new round's equipment during freeze time, it shifts effortlessly from fast play-by-play during action to deep, contextual analysis during the pause.
